```
    1: {"left": 0, "right": 2},
    2: {"left": 1, "right": 3},
    3: {"left": 2, "right": 3},

}

# Parameters
gamma = 0.9 # Discount factor
alpha = 0.1 # Learning rate
q_table = np.zeros((len(states), len(actions))) # Q-Table initialization # Training loop
for episode in range(1000):
    state = 0 # Start state
    while state != 3: # Terminal state action_idx = np.random.choice(len(actions)) action =
actions[action_idx]
        next_state = transitions[state][action]
        reward = rewards[next_state]

        # Update Q-value using Bellman equation best_next_q = np.max(q_table[next_state])
q_table[state, action_idx] += alpha *(reward + gamma* best_next_q - q_table[state, action_idx]) state
= next_state # Transition to the next state # Display Q-Table
print("Trained Q-Table:")
print(q_table)
```

Understanding the Example

1. **State and Action Representation**:
    - States are represented as integers (e.g., 0,1,2,30, 1, 2, 3).
    - Actions include "left" and "right," corresponding to index positions in the Q-Table.
2. **Rewards and Transitions**:
    - Each state-action pair transitions to a new state defined in the transitions dictionary.
    - The terminal state (state 33) offers the highest reward (11).
3. **Q-Table Initialization**:
    - The Q-Table is initialized with zeros, where rows represent states and columns represent actions.
4. **Training Loop**: